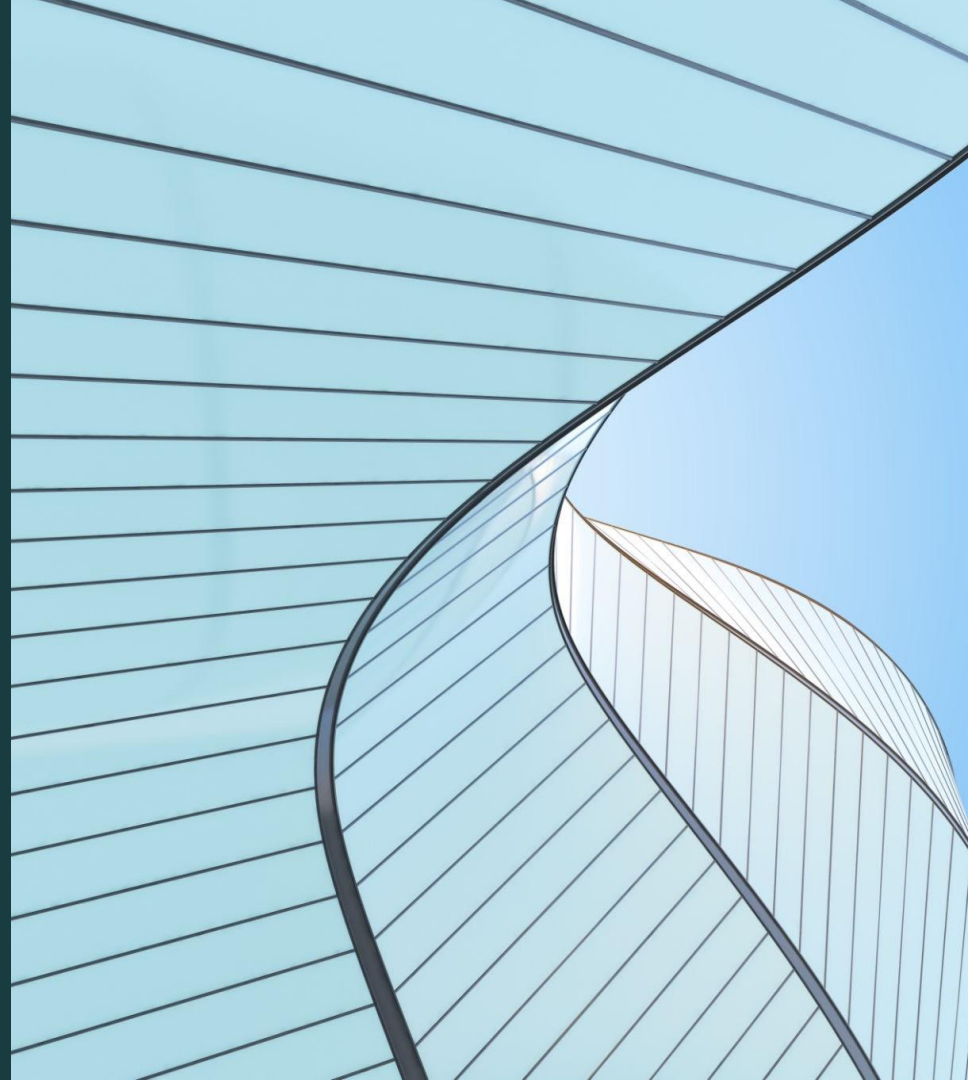


Calculation of Dimensionless Numbers for Water-in-Gas Droplet Formation

Isra Hussain CHEM 4699
Dr. Stockton, Augustine Debrah
24 April 2026



Introduction & Background

- Calculations regarding oil and water have been performed in the past
- Limit to the examination based on the visualization of droplet formation
- Quantitative research in terms of dimensionless numbers
 - Capillary, Weber, and Reynolds numbers
 - underdeveloped
- Accurate prediction on droplet size
 - help examine flow rates in both a gaseous and fluidic context.
- Python-based system
 - Gas flow rates
 - Liquid flow rates
 - Number of droplets
- Three-way relationship between the gas flow rate, liquid flow rates and number of droplets

Equations Utilized

- Volumetric flow rate conversion (gas)
- Volumetric flow rate conversion (liquid)
- Gas cross-sectional area (annulus)
- Hydraulic diameter
- Gas velocity
- Capillary number
- Weber number
- Reynolds number
- Droplet volume (from flow + count)
- Droplet diameter (sphere assumption)

Python Script

- Importing NumPy, pandas, and matplotlib, the numerical operations can be handled with python code accordingly
- Utilizing d_i function for each of the variables
 - d_i, d_o, D_i

```
# USER INPUTS
# This block collects experimental parameters from the user.
# IMPORTANT: These inputs must be in the units asked for, because we later
# convert them into SI (m^3/s, Pa*s, N/m, m, kg/m^3).
print("\n=== ENTER CONSTANT INPUT VALUES ===")

# Note: Gas flow rates (Qg), Liquid flow rates (Ql), and Droplet counts
# are now imported directly from the Excel file to prevent manual entry e

# Geometric diameters:
# d_i = liquid phase inner needle diameter (m) = 0.0003048
# d_o = outer diameter of the needle (for gas flow displacement)(m)= 0.00
# D_i = outlet tubing inner diameter (m)= 0.0024
d_i = float(input("Enter liquid phase inner needle diameter d in meters (m): "))
d_o = float(input("Enter liquid needle outer diameter d_o (m): "))
D_i = float(input("Enter outlet tubing inner diameter D in meters (m): "))

# Fluid properties: at 21 Degrees Celcius
# Gas (Continuous phase) - Nitrogen
# rho_g = gas density (kg/m^3) = 1.155
# mu_g = gas dynamic viscosity (Pa*s) = 17.63e-6
# sigma = Interfacial surface tension (N/m) = 0.0728 (air)
rho_g = float(input("Enter gas density (kg/m^3): "))
mu_g = float(input("Enter gas viscosity (Pa*s): "))
sigma = float(input("Enter surface tension (N/m): "))
```

Python Script

- Through calculations of dimensionless numbers,
 - Ca_l _list, We_l _list, Re_l _list
- Utilization of different equations for the conversions
- Using raw_df as the main DataFrame
 - store and process all variables

```
# UNIT CONVERSIONS & DIMENSIONLESS NUMBERS FUNCTIONS

# 1. SI Unit Conversions
raw_df['Qg_m3s'] = raw_df['Qg'] * 1e-3 / 60
raw_df['Ql_m3s'] = raw_df['Ql'] * 1e-6 / 60

# 2. Liquid Cross-sectional Area (Annulus)
# Liquid flows through the center of the needle (d_i)
A_l = (np.pi / 4) * (d_i**2)

# Gas Cross-sectional Area (gap between the dispensing needle OD (d_o) and outl
A_g = (np.pi / 4) * (D_i**2 - d_o**2)

# 3. Velocity Calculations (m/s)
raw_df['U_g'] = raw_df['Qg_m3s'] / A_g
raw_df['U_l'] = raw_df['Ql_m3s'] / A_l

# 4. Dimensionless Numbers
# For characteristic length, we typically use the Hydraulic Diameter of the ann
D_h = D_i - d_o

raw_df['Ca_g'] = (mu_g * raw_df['U_g']) / sigma
raw_df['We_g'] = (rho_g * (raw_df['U_g']**2) * D_h) / sigma
raw_df['Re_g'] = (rho_g * raw_df['U_g'] * D_h) / mu_g

# 5. Gas-to-Liquid Volumetric Ratio
raw_df['Flow_Ratio'] = raw_df['Qg_m3s'] / raw_df['Ql_m3s']
```

Reading Excel File

- Uploading Excel file file upload widget
- Reading raw data into Python using `pd.read_excel()`
- Assigning column names (Qg, Ql, T1, T2, T3)
- Cleaning data by removing empty rows and invalid values
- Converting all columns to numeric format for calculations
- Storing cleaned data in `raw_df` for further processing

```
# 4. Excel Data Processing
def on_button_clicked(b):
    global final_df
    global raw_df

    with output:
        clear_output()
        if not uploader.value:
            return

    try:
        # Load Raw Content
        try:
            uploaded_file = uploader.value[0]
            content = uploaded_file.content
        except KeyError:
            file_name = list(uploader.value.keys())[0]
            content = uploader.value[file_name].content
```

```
# Data Cleaning
raw_df = pd.read_excel(io.BytesIO(content), header=1)
raw_df.columns = ['Qg', 'Ql', 'T1', 'T2', 'T3']
raw_df['Qg'] = raw_df['Qg'].ffill()
raw_df = raw_df.dropna(subset=['Ql', 'T1'])
for col in raw_df.columns:
    raw_df[col] = pd.to_numeric(raw_df[col], errors='coerce')
raw_df = raw_df.dropna(subset=['Ql', 'T1'])
```

Gas Flow	Liq Flow	Count/min			Avg Count	Volume (mL)			Avg Vol	Diameter (mm)			Avg Dia	Dia std
(L/min)	(mL/min)	T1	T2	T3		T1	T2	T3	(mL)	T1	T2	T3	(mm)	
2	0.01	9	9	9	9	0.001111	0.001111	0.001111	0.001111	1.285	1.285	1.285	1.285	0
2	0.02	18	18	18	18	0.001111	0.001111	0.001111	0.001111	1.285	1.285	1.285	1.285	0
2	0.03	27	27	27	27	0.001111	0.001111	0.001111	0.001111	1.285	1.285	1.285	1.285	0
2	0.04	36	36	36	36	0.001111	0.001111	0.001111	0.001111	1.285	1.285	1.285	1.285	0
2	0.05	45	45	46	45.33	0.001111	0.001111	0.001087	0.001103	1.285	1.285	1.276	1.282	0.00542
2	0.06	54	54	55	54.33	0.001111	0.001111	0.001091	0.001104	1.285	1.285	1.277	1.282	0.00452
2	0.07	63	63	63	63	0.001111	0.001111	0.001111	0.001111	1.285	1.285	1.285	1.285	0
2	0.08	72	72	72	72	0.001111	0.001111	0.001111	0.001111	1.285	1.285	1.285	1.285	0
2	0.09	81	81	81	81	0.001111	0.001111	0.001111	0.001111	1.285	1.285	1.285	1.285	0
1.8	0.01	9	9	9	9	0.001111	0.001111	0.001111	0.001111	1.285	1.285	1.285	1.285	0
1.8	0.02	17	17	18	17.33	0.001176	0.001176	0.001111	0.001155	1.31	1.31	1.285	1.302	0.0143
1.8	0.03	25	25	26	25.33	0.0012	0.0012	0.001154	0.001185	1.318	1.318	1.301	1.313	0.00989
1.8	0.04	34	33	33	33.33	0.001176	0.001212	0.001212	0.0012	1.31	1.323	1.323	1.318	0.00756
1.8	0.05	41	42	41	41.33	0.00122	0.00119	0.00122	0.00121	1.326	1.315	1.326	1.322	0.00612
1.8	0.06	49	48	50	49	0.001224	0.00125	0.0012	0.001225	1.327	1.337	1.318	1.327	0.00903
1.8	0.07	56	55	56	55.67	0.00125	0.001273	0.00125	0.001258	1.337	1.345	1.337	1.339	0.00465
1.8	0.08	62	63	63	62.67	0.00129	0.00127	0.00127	0.001277	1.351	1.344	1.344	1.346	0.00415
1.8	0.09	68	68	69	68.33	0.001324	0.001324	0.001304	0.001317	1.362	1.362	1.356	1.36	0.00382
1.6	0.01	7	8	7	7.333	0.001429	0.00125	0.001429	0.001369	1.397	1.337	1.397	1.377	0.0351
1.6	0.02	16	15	16	15.67	0.00125	0.001333	0.00125	0.001278	1.337	1.366	1.337	1.346	0.0168
1.6	0.03	24	23	24	23.67	0.00125	0.001304	0.00125	0.001268	1.337	1.356	1.337	1.343	0.011

Python Script (Dimensionless Number Calculations)

- Flow rates to SI units
- Calculating gas cross-sectional area and hydraulic diameter
- Calculated values to a physical table
- Organize calculated values accordingly
- Formatting clearer results for visualization processes
- Avg_C, D, V, U_g, Ca_g, We_g, Re_g

```
# --- DIMENSIONLESS NUMBERS CALCULATIONS ---
# 1. SI Unit Conversions
raw_df['Qg_m3s'] = raw_df['Qg'] * 1e-3 / 60
raw_df['QL_m3s'] = raw_df['QL'] * 1e-6 / 60

# 2. Geometry Refinement
A_g = (np.pi / 4) * (D_i**2 - d_o**2)
D_h = D_i - d_o

# 3. Gas Velocity
raw_df['U_g'] = raw_df['Qg_m3s'] / A_g

# 4. Dimensionless Numbers
raw_df['Ca_g'] = (mu_g * raw_df['U_g']) / sigma
raw_df['We_g'] = (rho_g * (raw_df['U_g']**2) * D_h) / sigma
raw_df['Re_g'] = (rho_g * raw_df['U_g'] * D_h) / mu_g

# --- DIMENSIONLESS NUMBERS REPORT TABLE ---
# 5. Extract and Rename Columns (Added Avg_C and Avg_D)
physics_report = raw_df[['Qg', 'QL', 'Avg_C', 'Avg_D', 'Avg_V', 'U_g', 'Ca_g', 'We_g', 'Re_g']].copy()

physics_report = physics_report.rename(columns={
    'Qg': 'Qg (L/min)',
    'QL': 'QL (mL/min)',
    'Avg_C': 'Avg Count',
    'Avg_D': 'Avg Dia (mm)',
    'Avg_V': 'Droplet Vol (mL)',
    'U_g': 'Gas Velocity (m/s)',
    'Ca_g': 'Capillary (Ca_g)',
    'We_g': 'Weber (We_g)',
    'Re_g': 'Reynolds (Re_g)',
})

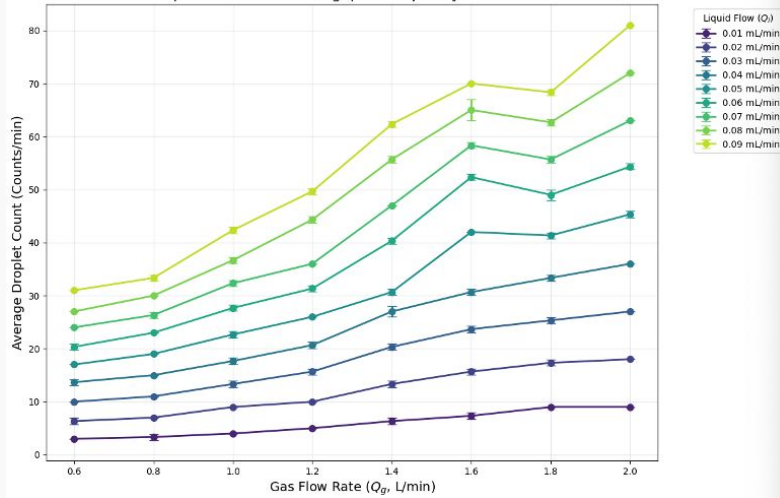
# 6. Apply Styling Engine (Added formatting for Count and Diameter)
styled_physics = physics_report.style.format({
    'Qg (L/min)': lambda x: fmt_sf(x, 2),
    'QL (mL/min)': lambda x: fmt_sf(x, 2),
    'Avg Count': lambda x: fmt_sf(x, 4),
    'Avg Dia (mm)': lambda x: fmt_sf(x, 4),
    'Droplet Vol (mL)': lambda x: fmt_sf(x, 4),
    'Gas Velocity (m/s)': lambda x: fmt_sf(x, 4),
    'Capillary (Ca_g)': lambda x: fmt_sf(x, 4),
    'Weber (We_g)': lambda x: fmt_sf(x, 4),
    'Reynolds (Re_g)': lambda x: fmt_sf(x, 4)
}).set_table_styles([
    {'selector': 'th', 'props': [('text-align', 'center'), ('background-color', '#f8f9fa'), ('border', '1px solid #ddd')]},
    {'selector': 'td', 'props': [('text-align', 'center'), ('border', '1px solid #eee')]}
]).hide(axis='index')

# Final Display
display(styled_physics)
```

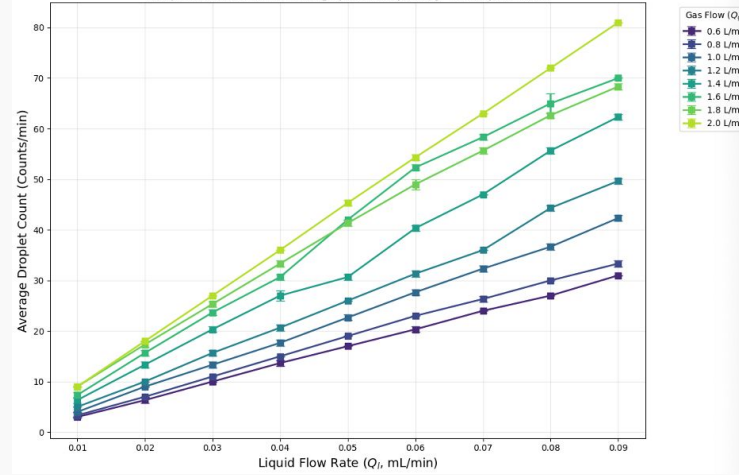

Numerical Plot and Input Values

Qg (L/min)	Ql (mL/min)	Avg Count	Avg Dia (mm)	Droplet Vol (mL)	Gas Velocity (m/s)	Capillary (Ca_g)	Weber (We_g)	Reynolds (Re_g)
2	0.01	9	1.285	0.001111	7.876	0.001907	1.762	923.9
2	0.02	18	1.285	0.001111	7.876	0.001907	1.762	923.9
2	0.03	27	1.285	0.001111	7.876	0.001907	1.762	923.9
2	0.04	36	1.285	0.001111	7.876	0.001907	1.762	923.9
2	0.05	45.33	1.282	0.001103	7.876	0.001907	1.762	923.9
2	0.06	54.33	1.282	0.001104	7.876	0.001907	1.762	923.9
2	0.07	63	1.285	0.001111	7.876	0.001907	1.762	923.9
2	0.08	72	1.285	0.001111	7.876	0.001907	1.762	923.9
2	0.09	81	1.285	0.001111	7.876	0.001907	1.762	923.9
1.8	0.01	9	1.285	0.001111	7.089	0.001717	1.427	831.5
1.8	0.02	17.33	1.302	0.001155	7.089	0.001717	1.427	831.5
1.8	0.03	25.33	1.313	0.001185	7.089	0.001717	1.427	831.5
1.8	0.04	33.33	1.318	0.0012	7.089	0.001717	1.427	831.5
1.8	0.05	41.33	1.322	0.00121	7.089	0.001717	1.427	831.5
1.8	0.06	49	1.327	0.001225	7.089	0.001717	1.427	831.5
1.8	0.07	55.67	1.339	0.001258	7.089	0.001717	1.427	831.5
1.8	0.08	62.67	1.346	0.001277	7.089	0.001717	1.427	831.5
1.8	0.09	68.33	1.36	0.001317	7.089	0.001717	1.427	831.5
1.6	0.01	7.333	1.377	0.001369	6.301	0.001526	1.128	739.1
1.6	0.02	15.67	1.346	0.001278	6.301	0.001526	1.128	739.1
1.6	0.03	23.67	1.343	0.001268	6.301	0.001526	1.128	739.1
1.6	0.04	30.67	1.356	0.001305	6.301	0.001526	1.128	739.1
1.6	0.05	42	1.315	0.00119	6.301	0.001526	1.128	739.1
1.6	0.06	52.33	1.299	0.001147	6.301	0.001526	1.128	739.1
1.6	0.07	58.33	1.318	0.0012	6.301	0.001526	1.128	739.1
1.6	0.08	65	1.33	0.001232	6.301	0.001526	1.128	739.1
1.6	0.09	70	1.349	0.001286	6.301	0.001526	1.128	739.1
1.4	0.01	6.333	1.446	0.001587	5.514	0.001335	0.8635	646.7
1.4	0.02	13.33	1.421	0.001502	5.514	0.001335	0.8635	646.7
1.4	0.03	20.33	1.413	0.001476	5.514	0.001335	0.8635	646.7

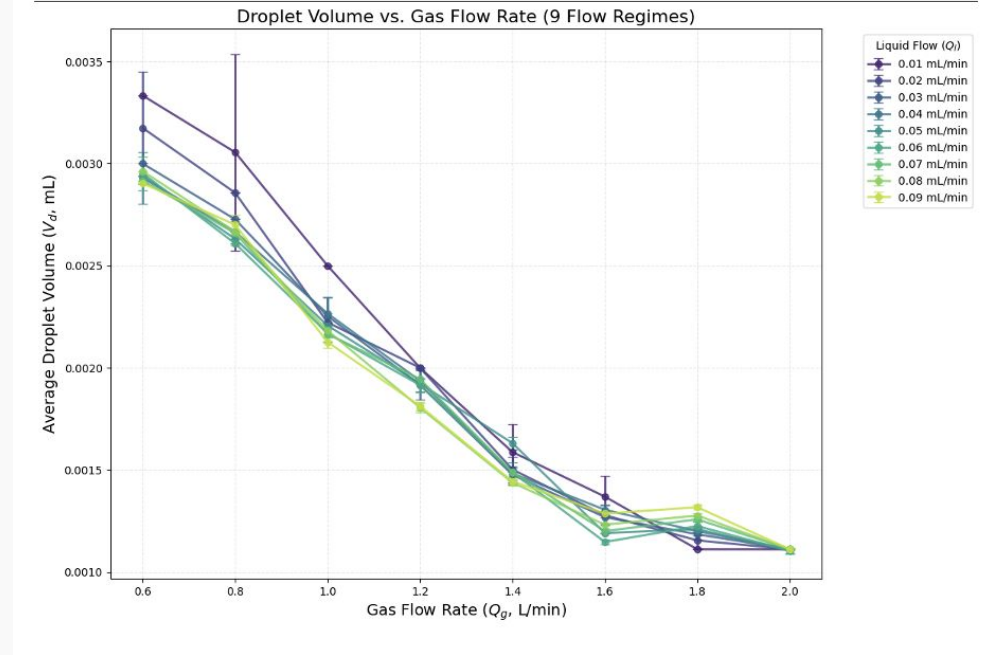
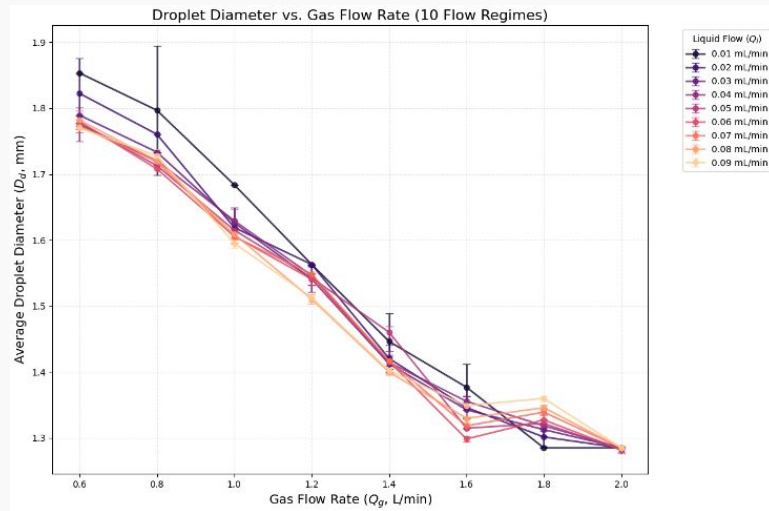
Droplet Production Throughput: Frequency vs. Gas Flow



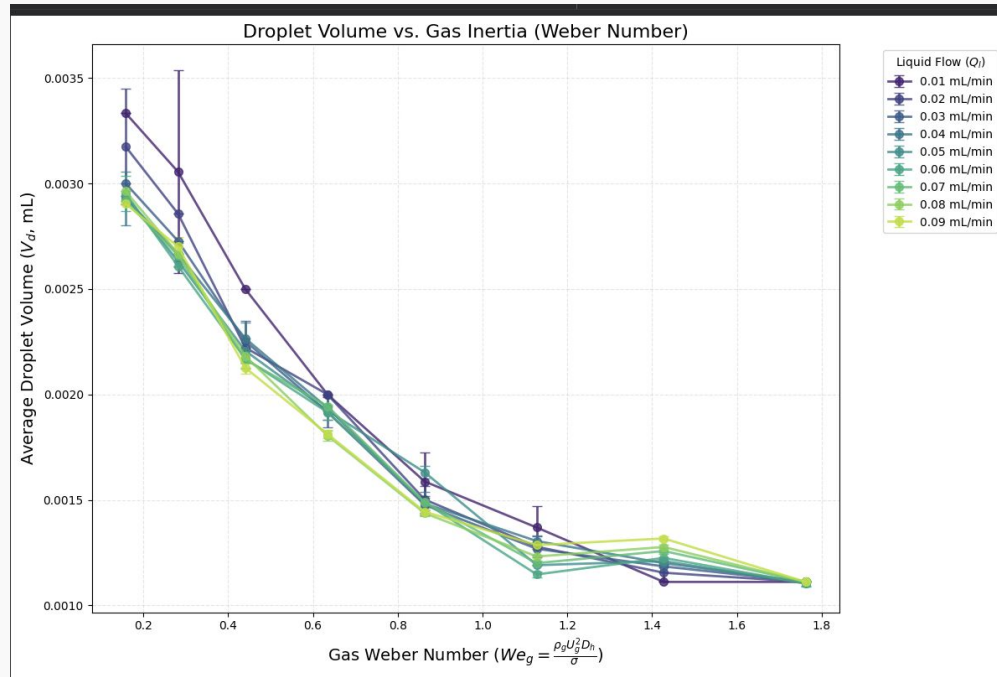
Droplet Production Throughput: Frequency vs. Liquid Flow



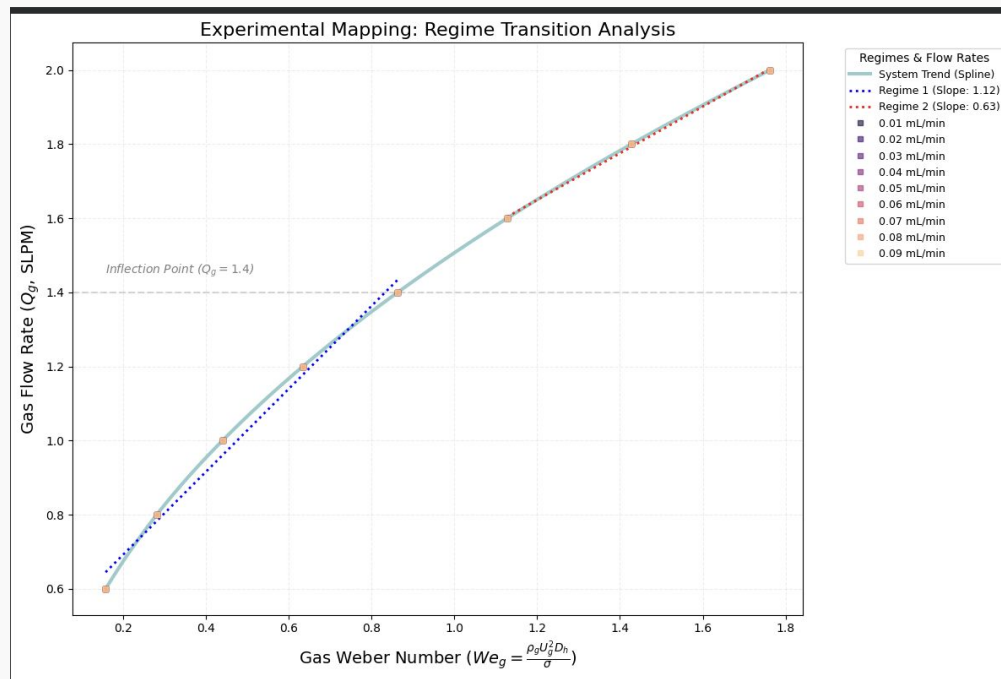
Droplet Production



Droplet Volume vs. Gas & Liquid Flow Rates

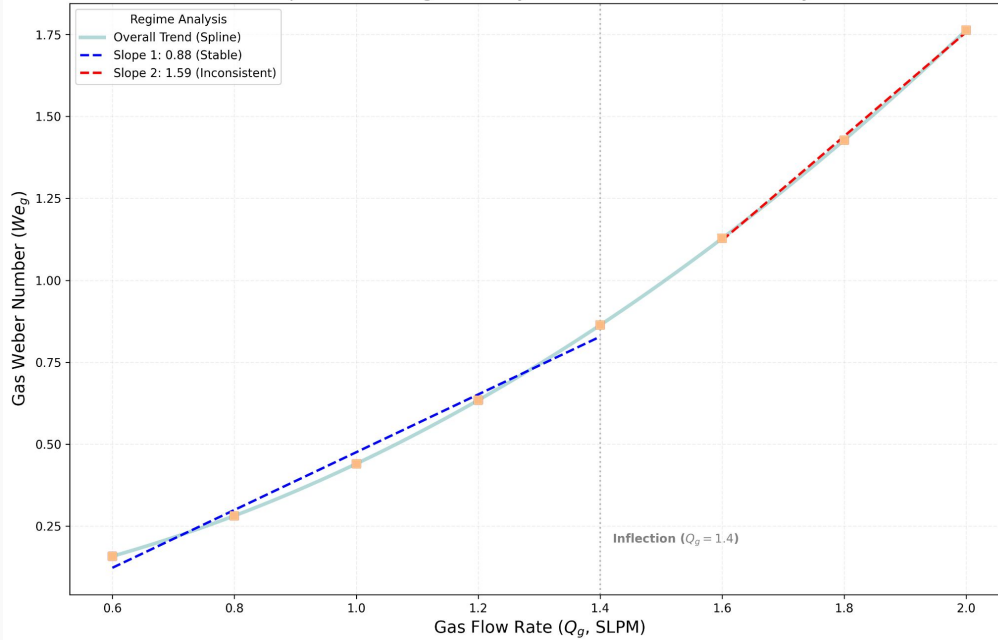


Droplet Volume vs. Gas Inertia



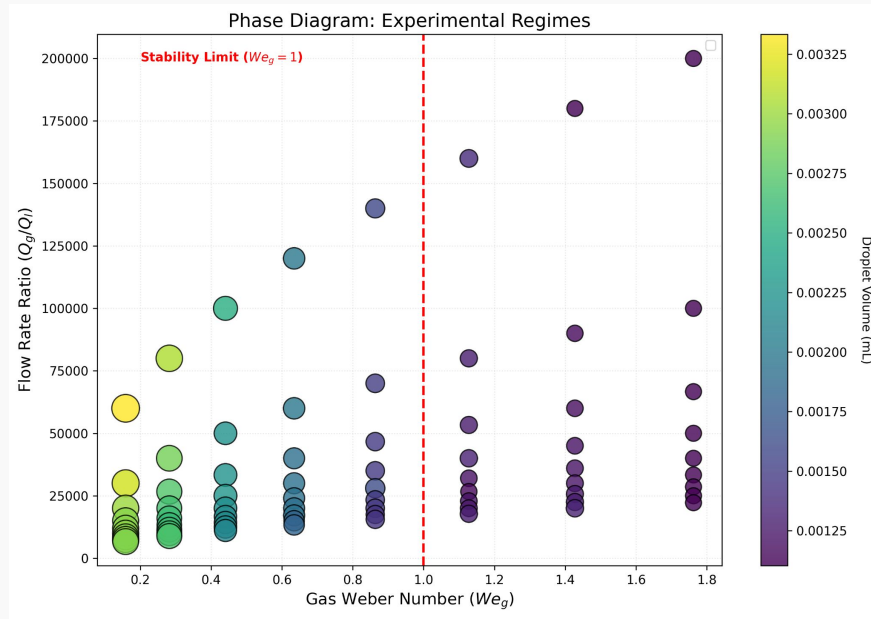
Experimental Mapping Analysis

Experimental Regime Analysis: Weber Number Sensitivity



- Weber number increase with flow rate
- Stronger inertial force
- Linear trend at lower force
- Steeper slope with higher flow rate
- Inflection point changes
- Inertial forces dominate at higher gas flows

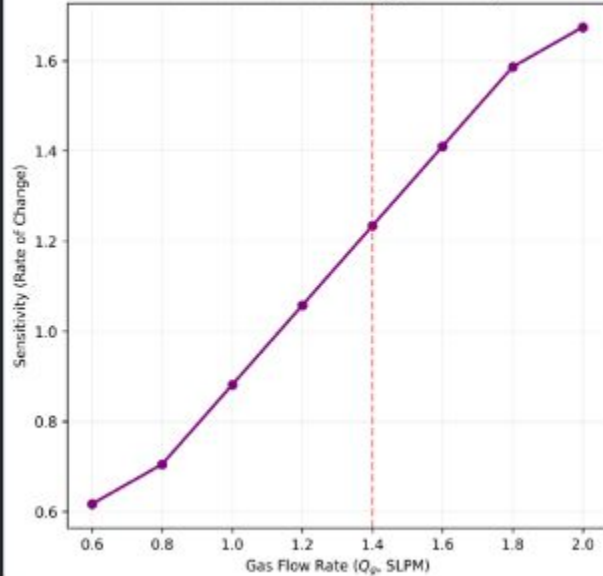
Weber Number Sensitivity



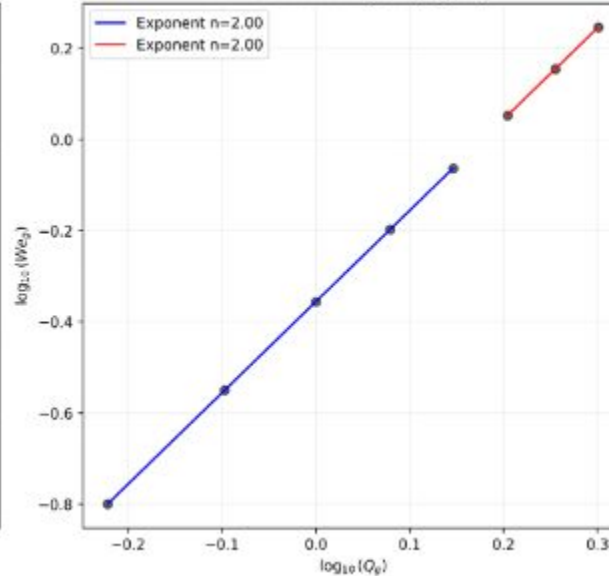
- Experimental Regimes
- Utilizing stabilizing limit in order to gauge the gas weber number changes
 - Differences in flow rate ratios

Phase Diagram: Experimental Regimes

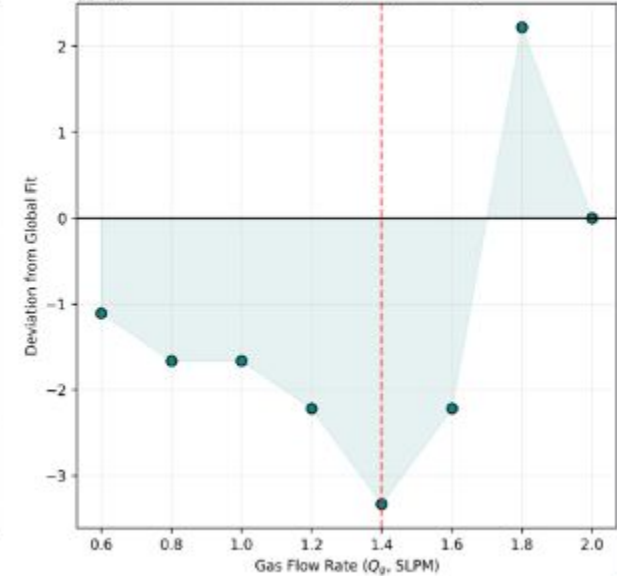
1. Momentum Sensitivity (Derivative)



2. Power Law Analysis (Log-Log)

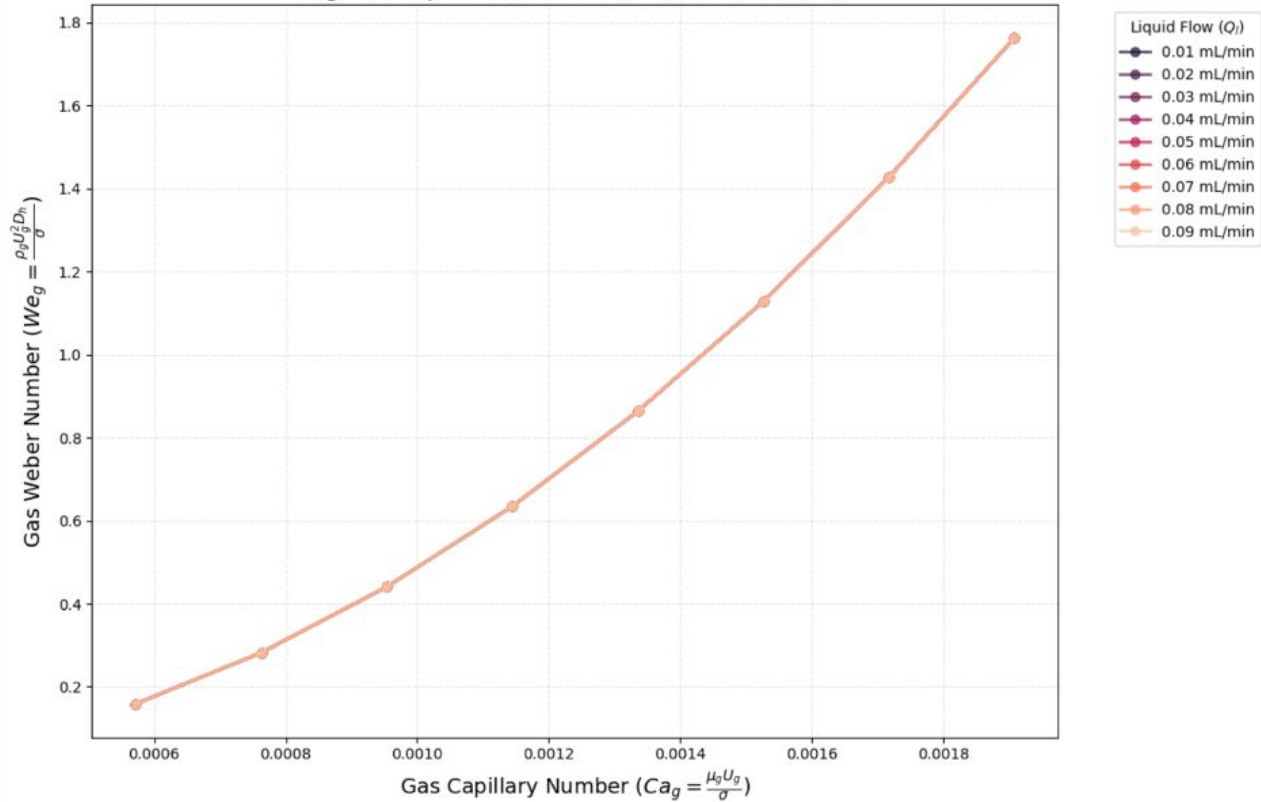


3. Residual Analysis (Instability)



Difference in Analysis
Processes for Gas Flow rate
and Log

Regime Map: Viscous vs. Inertial Force Balance



Regime Map

Discussion

- Computations and graphics provide insight into the analysis of hydrodynamic factors governing the process of droplets creation.
- Both Weber and Reynolds numbers are higher
 - Inertial forces have a greater effect on the process than surface tension
- Capillary number
 - Indicates the effect of viscous forces on the deformation of droplets
- 2D plots directly compare flow rates against the number of droplets, their volume, and diameter

Conclusion and Future Work

Python framework was designed to explore the correlation between flow rate of gases and liquids and droplet dynamics within the microfluidic environment

- Such analysis reveals the relationship between changes in flow conditions and droplets formed, their sizes and stability within conditions
- Computational workflow can be considered valid
- More information and data can be added for validation from other experimental setups
 - Accuracy of the input data used in the code can be improved through further adjustments in the parameters utilized during the calculations
- Physics behind droplet formation in microfluidics expansion